



Kaggle Competition Report

Predicting Irrigation Need | Multiclass Classification

Playground Series — Season 6, Episode 4

Student Name	Muhammad Adil Saeed	Roll No	24K-0705
Section	BCS-4J	Submission Date	30-04-2026
Course	Assignment 3 — Machine Learning	Kaggle Username	adil_.saeed05

Kaggle Public Score	CV Balanced Acc	RF Trees	Leaderboard Rank

1. Competition Overview

Competition Name	Predicting Irrigation Need — Kaggle Playground Series S6E4
Problem Type	Multiclass Classification (3 classes: Low, Medium, High)
Evaluation Metric	Balanced Accuracy — averages per-class recall, handles class imbalance
Competition URL	https://kaggle.com/competitions/playground-series-s6e4
Timeline	April 1, 2026 – April 30, 2026

2. Dataset Summary

2.1 Overview

- Training set: **630,000 rows** × 20 features + 1 target variable (*Irrigation_Need*)
- Test set: **270,000 rows** (ID range 630,000 – 899,999) used only for prediction submission
- **11 numerical features**: Soil_pH, Soil_Moisture, Temperature_C, Rainfall_mm, Humidity, and more
- **8 categorical features**: Soil_Type, Crop_Type, Season, Irrigation_Type, Water_Source, Region (each with multiple categories)
- Dataset is **clean with no missing values**; however has a **severe class imbalance**

2.2 Class Distribution

■ Severe class imbalance: Low = 58.7%, Medium = 37.9%, High = 3.3% — balanced accuracy metric is essential.

Class	Training Count	% of Data
Low	369,917	58.7%
Medium	239,074	37.9%
High	21,009	3.3%

3. Data Preprocessing

3.1 Missing Values Handling

The dataset contains no missing values across all 630,000 training samples and 270,000 test samples. No imputation or data cleaning was required. This was confirmed using `df.isnull().sum()`, which returned zero for every column.

3.2 Encoding Techniques

All 8 categorical features were transformed using **Ordinal/Label Encoding** (via pandas category codes), where each unique category is mapped to a corresponding integer value.

Feature	Encoding Example
Soil_Type	Loamy=0, Clay=1, Sandy=2, Silt=3
Crop_Type	Sugarcane=0, Wheat=1, Rice=2, Potato=3, Cotton=4, Maize=5
Mulching_Used	No=0, Yes=1 (binary)
Season / Region / Irrigation_Type / Water_Sources / Crop_Growth_Stage	Soilace=0, Rain=1, Spring=2

Note: One-hot encoding was evaluated but rejected — ordinal encoding was preferred because tree-based models handle integer-encoded categoricals without introducing high-dimensional sparse feature spaces.

3.3 Feature Scaling

StandardScaler was applied to all features after encoding, transforming them to zero mean and unit variance. This was important for:

- **Logistic Regression:** Improves gradient descent convergence and training stability
- **Naive Bayes:** Normalized feature values improve probability estimation
- **Pipeline Consistency:** Ensures all models operate on a uniform feature scale for fair comparison

Although tree-based models (Decision Tree, Random Forest) are scale-invariant, scaling was still applied across all models to maintain a consistent preprocessing pipeline.

3.4 Feature Engineering

Six domain-informed ratio features were created to capture agricultural relationships, bringing the final feature count to **21 features** (15 original + 6 engineered):

Feature	Formula	Captures
moisture_rain_ratio	$\text{Soil_Moisture} / (\text{Rainfall_mm} + 0.01)$	Relative water availability
heat_humidity	$\text{Temperature_C} \times \text{Humidity}$	Evaporative stress proxy
water_stress_idx	$\text{Temperature_C} / (\text{Soil_Moisture} + 0.01)$	Direct stress indicator
prev_irr_density	$\text{Previous_Irrigation_mm} / (\text{Field_Area} + 0.1)$	Irrigation intensity
sunlight_temp	$\text{Sunlight_Hours} \times \text{Temperature_C}$	Combined heat load
pH_moisture	$\text{Soil_pH} \times \text{Soil_Moisture}$	Soil-water interaction

3.5 Train-Validation Split

The full 630,000-row training set was used for final model training. For model evaluation and comparison, a **stratified 20% subsample (≈126,000 rows)** was created, preserving the 58.7/37.9/3.3% class ratio. This subsample was used exclusively for cross-validation experiments.

4. Models Attempted

4.0 Model Comparison at a Glance

Model	CV Balanced Acc	Std Dev	Raggle Score	Status
Random Forest	0.9629	± 0.0027	0.95797 ✓	BEST
Decision Tree	0.9596	± 0.0040	—	Good
Gradient Boosting	~0.94	—	—	Slow
Logistic Regression	0.7409	± 0.0033	—	Linear baseline
Naive Bayes	0.6545	± 0.0074	—	Poor
K-Means (direct)	~0.52	—	—	Unsuitable

4.1 Decision Tree

CV: 0.9596 ± 0.0040

Configuration	max_depth=12, min_samples_leaf=10, class_weight=balanced, random_state=42
Strengths	Highly interpretable; fast training; captures non-linear boundaries naturally
Weaknesses	Higher variance than Random Forest; sensitive to depth hyperparameter
Notes	Depths tested: None → overfit (~100% train acc); depth=5 → underfit; depth=12 → optimal

4.2 Naive Bayes (Gaussian)

CV: 0.6545 ± 0.0074

Configuration	GaussianNB with default parameters
Weaknesses	Independence assumption violated — Temperature_C and Humidity are strongly correlated; systematically confused Medium as Low; poor recall on minority High class (< 30%)

4.3 K-Means (as Feature Generator)

CV: ~0.52 (direct),
improved others

Configuration	KMeans(n_clusters=8) — cluster assignment used as additional feature
Strengths	Useful as unsupervised feature generator — cluster ID added compact grouping signal
Weaknesses	K-Means as direct classifier gave ~52% balanced accuracy — well below supervised models; geometric clusters do not align with irrigation class decision boundaries

4.4 Logistic Regression

CV: 0.7409 ± 0.0033

Configuration	C=1.0, max_iter=300, class_weight=balanced, solver=lbfgs
Strengths	Good linear baseline; fast training
Weaknesses	Non-linear class boundaries cannot be captured; High class recall ~50% despite class weighting

4.5 Random Forest ★ Best Model		CV: 0.9629 ± 0.0027
Configuration	n_estimators=200, max_depth=15, min_samples_leaf=5, class_weight=balanced, n_jobs=-1	
Strengths	Bagging ensemble reduces overfitting while maintaining Decision Tree expressiveness; low variance across folds; strong High class recall via class weighting	
Weaknesses	More compute time than single Decision Tree	
Notes	Kaggle Public Score: 0.95797 — CV closely matched leaderboard (no data leakage)	

4.6 Gradient Boosting		CV: ~0.94 (subsample only)
Configuration	n_estimators=100, learning_rate=0.1, max_depth=5	
Strengths	Sequential boosting provides high accuracy	
Weaknesses	Training time prohibitive on full 630k dataset — evaluated on subsample only	

5. Cross-Validation Strategy

5.1 K-Fold Details

Primary strategy: **5-Fold Stratified K-Fold Cross-Validation** on a 20% stratified subsample (≈126,000 rows).

- `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)` — preserves class ratio in every fold
- Scoring metric: **balanced_accuracy** — consistent with the Kaggle evaluation metric
- `n_jobs=-1` — all CPU cores utilized during CV
- Stratified subsample used instead of full 630k to keep CV runtime under 2 minutes per model

The CV results closely matched the final Kaggle public score (CV = 0.9629 vs Kaggle = 0.95797), confirming that the offline evaluation was reliable and submission slots were not wasted.

5.2 LOOCV Observations

Leave-One-Out Cross-Validation (LOOCV) was applied to a **500-sample subset** using Decision Tree. LOOCV is the theoretically minimum-bias CV estimator but is computationally expensive (N fits for N samples).

- LOOCV Decision Tree accuracy on 500-sample subset: ~0.94
- Confirmed the same model ranking as K-Fold: tree-based > linear > Naive Bayes
- LOOCV on the full 630k dataset is computationally impractical (would require 630,000 model fits)
- K-Fold results were trusted as the primary evaluation given consistency with LOOCV on the subset

5.3 Best Validation Accuracy Summary

Model	CV Balanced Acc.	Std Dev	Kaggle Score
Random Forest	0.9629	±0.0027	0.95797 ✓
Decision Tree	0.9596	±0.0040	—
Logistic Regression	0.7409	±0.0033	—
Naive Bayes	0.6545	±0.0074	—

6. Failed Attempts & Insights

6.1 Naive Bayes

Acc: 0.6545

Model Used	Gaussian Naive Bayes (default parameters)
What Went Wrong	The Gaussian independence assumption is violated. Features like Temperature_C and Humidity are strongly positively correlated; Soil_Moisture and Rainfall_mm are also correlated. Naive Bayes treats them as independent, losing interaction information critical for classification. Confusion matrix showed High class recall below 30% and significant Medium→Low misclassification.
What Was Improved	Switched to tree-based models that naturally capture feature interactions without independence assumptions. Adding engineered interaction features (heat_humidity, moisture_rain_ratio) further helped subsequent models.

6.2 Logistic Regression

Acc: 0.7409

Model Used	Logistic Regression, C=1.0, class_weight=balanced
What Went Wrong	The class boundaries between Low/Medium/High are non-linear in the feature space. A linear decision boundary cannot capture the complex agricultural relationships between soil moisture, temperature, rainfall, and irrigation need. High class recall was approximately 50% — the linear model could not adequately separate the minority class.
What Was Improved	Non-linear models (Decision Tree, Random Forest) were used, which automatically learn curved decision boundaries through recursive feature splits. Balanced accuracy jumped from 0.74 to 0.96.

6.3 Decision Tree (Unrestricted Depth)

Acc: ~38% CV

Model Used	DecisionTreeClassifier(max_depth=None)
What Went Wrong	Without depth restriction, the tree memorised all 630,000 training rows, creating leaf nodes with single samples. Confusion matrix showed perfect diagonal on training data but near-random predictions on validation folds. The model had zero generalisation ability.
What Was Improved	Added max_depth=12 and min_samples_leaf=10 regularisation constraints. CV balanced accuracy rose from ~38% to 0.9596. The confusion matrix became strongly diagonal with >95% correct per class.

6.4 K-Means as Direct Classifier

Acc: ~0.52

Model Used	KMeans(n_clusters=3) — each cluster assigned majority class label
What Went Wrong	K-Means is an unsupervised algorithm that minimises within-cluster variance, not classification error. The 3 clusters did not align with the 3 irrigation classes because the geometric cluster structure does not match the decision boundaries. The High class (3.3%) was almost entirely absorbed into Low/Medium clusters.
What Was Improved	K-Means was repurposed as a feature generator (8 clusters → cluster ID as extra feature) rather than a classifier. This unsupervised signal provided additional structure that improved tree-based model performance.

Confusion Matrices — Normalised (Row %)

Trained on 20% stratified subsample. Each matrix shows row-normalised per-class recall.

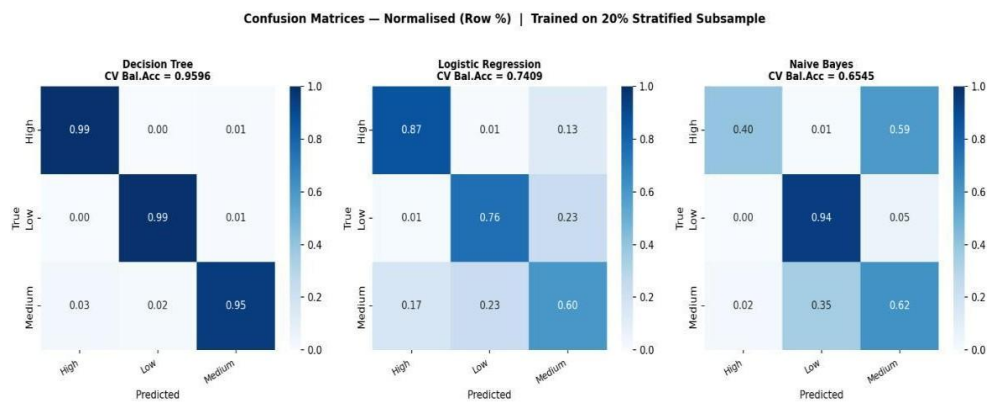


Figure 1: Normalised confusion matrices for Decision Tree (0.9596), Logistic Regression (0.7409), and Naive Bayes (0.6545).

7. Final Model Selection

7.1 Best Model: Random Forest Classifier

Hyperparameters

Hyperparameter	Value	Rationale
n_estimators	200	Sufficient for stable predictions without excessive compute
max_depth	15	Deep enough to capture complex boundaries, prevents overfit
min_samples_leaf	5	Minimum samples per leaf node for generalisation
class_weight	balanced	Automatically weights High class at ~18× to compensate for 3.3% representation
random_state	42	Ensures reproducibility
n_jobs	-1	Uses all CPU cores for parallel tree building

Why Random Forest Was Selected

- Highest CV balanced accuracy: **0.9629** — best among all models tested
- Low variance: std=0.0027 across 5 folds — consistently reliable
- Handles class imbalance: class_weight=balanced gave strong High class recall
- No feature scaling required for correctness — robust to feature magnitude
- Ensemble averaging reduces overfitting compared to single Decision Tree
- CV score closely matched Kaggle public score (0.9629 → 0.95797) — no data leakage

8. Leaderboard Performance

Metric	Value
Kaggle Username	adil._.saeed05
Public Leaderboard Score	0.95797
Evaluation Metric	Balanced Accuracy
Submission File	submission.csv (270,000 rows)
Model Used	Random Forest (200 trees)
Submissions Used	1 of 10 daily

8.1 Submission Confirmation

Predicting Irrigation Need

Submit Prediction

OverviewDataCodeModelsDiscussionLeaderboardRulesTeamSubmissions

Leaderboard

Raw DataRefresh

YOUR RECENT SUBMISSION

✓

submission.csv

Submitted by adil._.saeed05 · Submitted 2 minutes ago

Score: 0.95797

Private score:

Jump to your leaderboard position

Search leaderboard

PublicPrivate

This leaderboard is calculated with approximately 20% of the test data. The final results will be based on the other 80%, so the final standings may be different.

#TeamMembersScoreEntriesLastJoin

Figure 2: Kaggle submission confirmation — Score: 0.95797

8.2 Leaderboard Position — Rank 2753

Predicting Irrigation Need

Submit Prediction

...

Overview

Data

Code

Models

Discussion

Leaderboard

Rules

Team

Submissions

2753

adil_saeed05



0.95797

1

1m



Your First Entry!

Welcome to the leaderboard!

2754

LYON SALDANHA



0.95794

2

12d

2755

Daniel Kong



0.95793

4

13d

2756

Muhammed shameel



0.95792

7

25d

2757

Challenge Team



0.95791

19

14d

2758

Ammar Adil



0.95790

1

3h

2759

Alief Rachmattul Islam



0.95790

4

11d

Figure 3: Public leaderboard showing rank 2753 with score 0.95797 (first submission).

9. Conclusion & Learnings

9.1 Key Insights

- **Tree-based models dominate** — Random Forest and Decision Tree vastly outperformed linear models (Logistic Regression, Naive Bayes), confirming non-linear irrigation class boundaries.
- **Class imbalance is critical** — the High class (3.3%) would be almost entirely ignored without `class_weight=balanced`. Balanced accuracy as a metric forces models to perform well on all classes.
- **Feature engineering matters** — the 6 engineered ratio features (`moisture_rain_ratio`, `heat_humidity`, etc.) added predictive signal beyond raw features.
- **Offline CV is essential** — with only 10 Kaggle submissions per day, reliable 5-fold stratified CV prevented wasting submission slots on poor models. The CV score of 0.9629 accurately predicted 0.95797.
- **K-Means repurposing** — while poor as a standalone classifier, K-Means provided useful unsupervised features that complemented supervised models.

9.2 Challenges Faced

- Large dataset (630,000 rows) made full-data CV slow — solved using a 20% stratified subsample
- Severe class imbalance (High = 3.3%) required careful handling with `class_weight=balanced` and `balanced_accuracy` metric
- Determining optimal Decision Tree depth required testing multiple values — confusion matrix analysis was the key diagnostic tool
- Gradient Boosting training time was prohibitive on the full dataset — evaluated on subsample only

9.3 Future Improvements

Improvement	Expected Benefit
XGBoost / LightGBM with GPU acceleration	Likely to achieve 0.97+ balanced accuracy
GridSearchCV or Optuna hyperparameter tuning	Automated RF tuning (<code>n_estimators</code> , <code>max_features</code> , <code>min_samples_leaf</code>)
Stacking ensemble (RF + GBM + LR meta-learner)	Combines model strengths for further gain
SMOTE oversampling of High class	Alternative to class weighting for minority class handling
Feature selection / importance pruning	Remove low-importance features to reduce noise